

Implementing a Simple Backtest

Let's implement a basic backtest for our moving average crossover strategy:

```
1. # Calculate strategy returns
2. apple['Strategy_Returns'] = apple['Signal'].shift(1) * apple['Close'].pct_change()
3.
4. # Calculate cumulative returns
5. apple['Cumulative_Strategy_Returns'] = (1 + apple['Strategy_Returns']).cumprod()
6. apple['Cumulative_Buy_Hold_Returns'] = (1 + apple['Close'].pct_change()).cumprod()
7.
8. # Plot performance comparison
9. plt.figure(figsize=(12, 6))
10. plt.plot(apple.index, apple['Cumulative_Strategy_Returns'], label='Strategy Returns')
11. plt.plot(apple.index, apple['Cumulative_Buy_Hold_Returns'], label='Buy & Hold Returns')
12. plt.title('Strategy Performance vs. Buy & Hold')
13. plt.xlabel('Date')
14. plt.ylabel('Cumulative Returns')
15. plt.legend()
16. plt.grid(True)
17. plt.show()
18.
19. # Calculate key metrics
20. total_return = apple['Cumulative_Strategy_Returns'].iloc[-1] - 1
21. annualized_return = ((1 + total_return) ** (252 / len(apple))) - 1
22. sharpe_ratio = np.sqrt(252) * (apple['Strategy_Returns'].mean() /
apple['Strategy_Returns'].std())
23.
24. print(f"Total Return: {total_return:.2%}")
25. print(f"Annualized Return: {annualized_return:.2%}")
26. print(f"Sharpe Ratio: {sharpe_ratio:.2f}")
27.
28.
```

This simple backtest:

1. Calculates daily returns for our strategy (by multiplying yesterday's signal by today's price change)